Assignment: Instruction Tuning and Direct Preference Optimization on a Small Language Model

Course Staff
Lead TA: Ahmad Jawwad

Abstract

This assignment introduces the post-training pipeline used to turn a pretrained language model into a more useful assistant. You will work with a small pretrained model, `Qwen/Qwen2.5-0.5B`, and train it in two stages: supervised fine-tuning (SFT) on instruction-response demonstrations, followed by Direct Preference Optimization (DPO) on pairwise preference data. You will implement the main SFT and DPO training components yourself, run the experiments on Kaggle T4 GPUs, generate responses on a fixed AlpacaEval subset, and evaluate model behavior using pairwise judgments from Kaggle model APIs. The goal is not to produce a state-of-the-art assistant, but to understand the mechanics, tradeoffs, and evaluation challenges of modern LLM alignment.

Contents

1 Learning Objectives **4**

2 High-Level Background **4**
2.1 Pretraining . . . 4
2.2 Supervised Fine-Tuning . . . 4
2.3 RLHF . . . 5
2.4 Direct Preference Optimization . . . 5
2.4.1 Why DPO Needs a Reference Model . . . 5
2.4.2 The Loss, Intuitively . . . 5
2.4.3 Step-by-Step DPO Training . . . 6

3 What You Will Build **7**

4 Private Evaluation Interface **8**
4.1 Submission Adapter Contract . . . 8
4.2 Public Adapter Tests . . . 9

5 Compute Environment **9**
5.1 Kaggle Accelerator . . . 10
5.2 Dependency Guidance . . . 10
5.3 Starter Repository . . . 10

5.4 Kaggle Workflow . . . 11
5.5 Saving Work While Kaggle Jobs Run . . . 11

6 Model and Data 12
6.1 Model . . . 12
6.2 SFT Data . . . 12
6.3 Preference Data . . . 12
6.4 Evaluation Prompts . . . 13

7 Task 1: Repository and Kaggle Setup 13

8 Task 2: Baseline Generation 13

9 Task 3: Supervised Fine-Tuning 14
9.1 Implementation Requirements . . . 14
9.2 Suggested Hyperparameters . . . 15

10 Task 4: Two From-Scratch DPO Runs 15
10.1 Implementation Requirements . . . 15
10.2 Required DPO Runs . . . 16
10.3 Reference Training Recipe . . . 16
10.4 Suggested Hyperparameters . . . 17
10.5 Metrics . . . 17

11 Task 5: Idiot Sandwich — TRL DPOTrainer With LoRA Persona Tuning 18

12 Task 6: Generate Evaluation Outputs 18

13 Task 7: Evaluation With Kaggle Models 19
13.1 Kaggle Benchmark Setup . . . 19
13.2 Evaluator Reliability Requirements . . . 20

14 Task 8: Final Report 20
14.1 Repository, Models, and Data . . . 21
14.2 Experimental Configuration . . . 21
14.3 Parameter Choices and Deviations . . . 21
14.4 Training Behavior . . . 21
14.5 DPO-versus-SFT Results . . . 21
14.6 Interpretation and Error Analysis . . . 22
14.7 Limitations and Reproducibility . . . 22

15 Expected Runtime 22

16 Submission Checklist **23**

17 Academic Integrity and Collaboration **23**

18 Resources **23**

1 Learning Objectives

By the end of this assignment, you should be able to:

- Explain the distinction between pretraining, supervised fine-tuning, RLHF, and DPO.
- Train a causal language model with supervised fine-tuning while computing loss only on assistant response tokens.
- Implement the from-scratch DPO loss from pairwise preference data, then compare it with a TRL/LoRA DPO workflow.
- Train a small language model under realistic GPU and disk constraints.
- Design and run an evaluation pipeline based on fixed prompts, saved generations, and pairwise model judgments.
- Analyze alignment results critically, including failure cases, noisy judgments, and possible alignment tax.

2 High-Level Background

2.1 Pretraining

Modern language models are first trained with a self-supervised next-token prediction objective. Given a sequence of tokens $x_1, \ldots, x_T$, the model learns to maximize

$$\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}). \tag{1}$$

This stage teaches the model broad linguistic, factual, and reasoning patterns from large text corpora. However, a pretrained base model is not necessarily a good assistant. It may continue text, imitate undesired formats, fail to follow instructions reliably, or produce unsafe content.

In this assignment, you will *not* pretrain a model. You will start from a pretrained base model, `Qwen/Qwen2.5-0.5B`, and study post-training.

2.2 Supervised Fine-Tuning

Supervised fine-tuning (SFT), also called instruction tuning, trains the model on examples of the form:

$$\texttt{instruction} \rightarrow \texttt{response}.$$

The model is trained with teacher forcing to assign high probability to the target response. A key implementation detail is that the loss should usually be applied only to response tokens, not to the prompt tokens. If x is the instruction prompt and y is the target response, the SFT objective is

$$\mathcal{L}_{\text{SFT}}(\theta) = -\log \pi_\theta(y \mid x). \tag{2}$$

SFT teaches a base model to follow an instruction-response format and to imitate high-quality demonstrations.

2.3 RLHF

Reinforcement Learning from Human Feedback (RLHF) typically involves three stages:

1. Train an SFT model from demonstrations.
2. Train a reward model from pairwise human preferences.
3. Optimize the language model policy against the reward model, often using PPO with a KL penalty.

RLHF is powerful but complex. It requires reward modeling, online sampling, reinforcement learning machinery, and careful stabilization. Within this broad framework, this assignment focuses on two pieces: supervised fine-tuning (SFT) and policy optimization. For the policy-optimization stage, you will use Direct Preference Optimization (DPO) instead of PPO.

You will *not* implement PPO or a full reward-model-based RLHF pipeline in this assignment. Instead, you will implement DPO, a simpler preference-learning method.

2.4 Direct Preference Optimization

Direct Preference Optimization (DPO) is a preference-learning method introduced by Rafailov et al. in *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. It is designed to get many of the benefits of PPO-style policy optimization while avoiding the most complicated parts of a PPO-style reinforcement learning pipeline.

The data for DPO is pairwise. Each training example contains:

$$\texttt{instruction}\ x,\ \texttt{chosen response}\ y_w,\ \texttt{rejected response}\ y_l.$$

The chosen response is the answer a human or preference dataset judged better. The rejected response is a worse answer to the same instruction. DPO does not ask the model to imitate only the chosen response. Instead, it asks the model to make the chosen response *more likely than the rejected response*, while staying close to a frozen reference model.

2.4.1 Why DPO Needs a Reference Model

If we only pushed the policy to prefer chosen responses over rejected responses, the model could drift too far from the SFT model. That can produce strange outputs, repetition, overconfident refusals, or loss of general helpfulness. DPO therefore compares two models:

- the **policy model** π_θ, which is trainable;
- the **reference model** π_{ref}, which is frozen and usually initialized from the same SFT checkpoint.

The reference model acts like an anchor. DPO rewards the policy for improving the chosen-vs-rejected preference *relative to what the SFT model already believed*. This is the part that replaces the explicit KL penalty used in many RLHF pipelines.

2.4.2 The Loss, Intuitively

For each pair, compute four response log probabilities:

Quantity	Meaning
$\log \pi_\theta(y_w \mid x)$	policy log probability of the chosen response
$\log \pi_\theta(y_l \mid x)$	policy log probability of the rejected response
$\log \pi_{\text{ref}}(y_w \mid x)$	reference log probability of the chosen response
$\log \pi_{\text{ref}}(y_l \mid x)$	reference log probability of the rejected response

Then compare two preference gaps:

$$\Delta_\theta = \log \pi_\theta(y_w \mid x) - \log \pi_\theta(y_l \mid x), \tag{3}$$

$$\Delta_{\text{ref}} = \log \pi_{\text{ref}}(y_w \mid x) - \log \pi_{\text{ref}}(y_l \mid x). \tag{4}$$

Δ_θ says how much the trainable model prefers the chosen response over the rejected response. Δ_{ref} says how much the original SFT model already preferred the chosen response. DPO trains the policy so that Δ_θ becomes larger than Δ_{ref}.

The per-example DPO loss is:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \left[(\log \pi_\theta(y_w \mid x) - \log \pi_\theta(y_l \mid x)) - (\log \pi_{\text{ref}}(y_w \mid x) - \log \pi_{\text{ref}}(y_l \mid x))\right]\right). \tag{5}$$

Equivalently:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \left[\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right]\right). \tag{6}$$

The hyperparameter β controls the strength of the reference-model constraint in the DPO formulation. A larger β penalizes deviation from the reference policy more strongly, while a smaller β permits greater deviation. It also scales the logits inside the logistic loss, so changing it changes the optimization dynamics as well as the effective regularization. In this assignment, $\beta = 0.1$ is the recommended starting point.

DPO in One Sentence

DPO teaches the model to increase the probability gap between preferred and rejected answers, measured relative to a frozen SFT reference model, without training a separate reward model or running online reinforcement learning.

2.4.3 Step-by-Step DPO Training

A single DPO update looks like this:

1. Format the instruction x exactly the same way for both the chosen and rejected response.
2. Tokenize the prompt-response sequence for y_w and for y_l.
3. Mask prompt tokens so that log probabilities are summed only over response tokens.
4. Run the trainable policy model on the chosen and rejected sequences.
5. Run the frozen reference model on the same chosen and rejected sequences with gradients disabled.
6. Sum token log probabilities over the response span for each sequence.

7. Compute the policy preference gap Δ_θ and the reference preference gap Δ_{ref}.
8. Apply the DPO logistic loss $-\log \sigma(\beta(\Delta_\theta - \Delta_{\text{ref}}))$.
9. Backpropagate only through the policy model.
10. Repeat with gradient accumulation, validation, and checkpointing.

The important implementation idea is that DPO is still ordinary supervised optimization over saved data. There is no environment, no online sampling loop, and no separate reward-model training stage.

3 What You Will Build

You will build a complete small-model alignment pipeline with three preference-optimization tracks:

1. Create a reproducible GitHub repository and Kaggle workflow.
2. Generate baseline responses from `Qwen/Qwen2.5-0.5B` on fixed evaluation subsets.
3. Train one SFT model on 30,000 instruction-response examples.
4. Run from-scratch DPO on Anthropic Helpful–Harmless-style preference data, then test it on the synthetic helpful/harmless evaluation set provided with the assignment.
5. Run from-scratch DPO on UltraFeedback-style preference data, then test it on the fixed AlpacaEval subset using the same seed across all models.
6. Run a TRL `DPOTrainer` + LoRA experiment on the provided persona preference subset in `data/persona_lora/ramsay_style_preferences.json`.
7. Generate outputs, build pairwise comparison files, and evaluate with model-based judgments.
8. Submit a complete written report comparing the SFT model, both from-scratch DPO models, and the LoRA-DPO model.

Library Boundary

You may use Hugging Face Transformers to load models and tokenizers. You may use PyTorch utilities such as `DataLoader`, optimizers, schedulers, and AMP. For the SFT implementation and the two from-scratch DPO runs, you may **not** use Hugging Face `Trainer`, TRL `DPOTrainer`, or a prebuilt DPO training loop. The only exception is the LoRA persona task, where you **should** use TRL's `DPOTrainer` with PEFT/LoRA.

Guide Configurations, Not Sacred Hyperparameters

The example sizes and hyperparameters in this manual are the reference recipe used to make the assignment feasible on Kaggle T4 GPUs. They are not guaranteed to be optimal. Students may try other learning rates, optimizers, batch sizes, data mixtures, LoRA ranks, or training lengths if they document the changes and compare results fairly.

4 Private Evaluation Interface

The course staff may use a private grader to check important implementation details, such as response-token masking, log-probability computation, and DPO loss correctness. Students will not receive direct access to this grader.

Your repository should still be organized clearly enough that staff can inspect and run your implementation. In particular, make sure your training code is easy to locate, your notebooks use reproducible paths, and your report explains any deviations from the guide configuration.

4.1 Submission Adapter Contract

To make private implementation checks possible without requiring everyone to use the same internal code layout, include a root-level file named `submission_adapter.py`. This file should expose a small set of wrapper functions that call into your own implementation.

```
def build_sft_batch(tokenizer, prompts, responses, max_length): ...

def compute_response_logprobs(model, tokenizer, prompts, responses, max_length): ...

def compute_sft_loss(model, tokenizer, prompts, responses, max_length): ...

def build_dpo_batch(
    tokenizer,
    prompts,
    chosen_responses,
    rejected_responses,
    max_length,
): ...

def compute_dpo_loss_from_logps(
    policy_chosen_logps,
    policy_rejected_logps,
    reference_chosen_logps,
    reference_rejected_logps,
    beta,
): ...

def compute_dpo_loss(
    policy_model,
    reference_model,
    tokenizer,
    prompts,
    chosen_responses,
    rejected_responses,
    beta,
    max_length,
): ...
```

The private checker may call these functions with small toy tokenizers and toy causal language models. These checks are separate from model-performance evaluation and are intended to verify that the core training math is implemented correctly.

Expected behavior:

- Pad shorter examples and truncate longer examples to `max_length`.
- Mask prompt and padding tokens with `-100` for supervised fine-tuning labels.
- Append an EOS token to each response before applying the sequence-length truncation when the tokenizer provides one. The EOS token may therefore be removed if the response is truncated at `max_length`.
- Compute log probabilities over response tokens only, not prompt or padding tokens.
- For `compute_sft_loss`, return a scalar response-only loss using either of two standard reductions: the mean of per-example summed response negative log probabilities, or the mean over all unmasked response tokens in the batch. Use the same reduction consistently and document it in your report.
- Preserve the input row order in all returned tensors.
- Return `chosen` and `rejected` LM batches from `build_dpo_batch`; each batch must contain `input_ids`, `attention_mask`, and `labels`.
- Return the scalar batch-mean DPO objective from `compute_dpo_loss_from_logps`.
- Keep reference-model computations under `no_grad` for DPO.

4.2 Public Adapter Tests

The starter repository includes a small public smoke-test suite in `tests/test_public_adapter_smoke.py` and lightweight test doubles in `tests/toy_lm.py`. These tests check the required function signatures and a few basic properties of SFT/DPO batching, response-only scoring, scalar losses, the tensor-level DPO objective, and gradient flow. They run entirely offline and do not require a GPU, a Hugging Face model download, or the assignment datasets.

Run them from the repository root:

```
python -m pip install -q pytest
PYTHONPATH=. python -m pytest -q tests/test_public_adapter_smoke.py
```

The provided `submission_adapter.py` is intentionally a scaffold and initially raises `NotImplementedError`. Replace each function body with a thin call into your own implementation while keeping the documented signatures unchanged.

Public Tests Are a Minimum Check

Passing the public tests does not guarantee full credit on the private implementation checks. The public suite exposes common mistakes without publishing the staff reference implementation or the exact numerical and edge-case checks used by the grader. You should also test truncation boundaries, unusual tokenizer configurations, mixed sequence lengths, row ordering, and small optimization steps in your own code.

5 Compute Environment

5.1 Kaggle Accelerator

Use a Kaggle notebook with the `GPU T4 x2` accelerator. You should verify that two GPUs are visible:

```
nvidia-smi
```

A typical output should show two Tesla T4 GPUs with roughly 15 GiB of memory each.

5.2 Dependency Guidance

Kaggle already includes a CUDA-compatible PyTorch build. Do not reinstall PyTorch unless you have a specific reason. A safe install pattern is:

```
python -m pip uninstall -y -q torchao || true
python -m pip install -q "transformers>=4.43.0,<5" "datasets>=2.18,<5" \
  huggingface_hub accelerate tqdm pyyaml sentencepiece safetensors pytest
python -m pip install -q -e . --no-deps
```

This setup is useful because Kaggle's preinstalled PyTorch is already matched to the notebook GPU runtime. Reinstalling PyTorch can waste time, pull the wrong CUDA build, or break GPU access. The optional torchao uninstall avoids a known compatibility issue where older Kaggle torchao builds can conflict with PEFT/LoRA loading. Installing the starter repo with pip install -e . –no-deps makes local imports work without letting pip replace the carefully managed Kaggle environment.

Your repository should include a minimal `requirements.txt` and `pyproject.toml`. The requirements should not pin `torch`. A suitable `requirements.txt` is:

```
transformers>=4.43.0,<5
datasets>=2.18,<5
huggingface_hub
accelerate
tqdm
pyyaml
sentencepiece
safetensors
pytest>=8
```

5.3 Starter Repository

The student-facing starter repository is available at github.com/ahmad903501/small-model-alignment-starter. It provides a clean, tested starting point without exposing the staff/reference solution code or completed SFT and DPO implementations.

The starter branch should contain only the essentials:

- `requirements.txt`;
- `pyproject.toml`;
- fixed data files under `data/`;
- fixed evaluation files under `eval_sets/`;
- the root-level `submission_adapter.py` scaffold;

- public adapter smoke tests and their toy model under `tests/`;
- the assignment PDF.

A minimal starter layout is:

```
small-model-alignment-starter/
  requirements.txt
  pyproject.toml
  submission_adapter.py
  tests/
    test_public_adapter_smoke.py
    toy_lm.py
  data/
    persona_lora/
      ramsay_style_preferences.json
      README.md
  eval_sets/
    hh_rlhf_eval_300_seed42.json
  docs/
    assignment_manual.pdf
```

Starter Repo Sanity Check

Before sharing the starter branch, clone it into a fresh environment and verify that the setup commands run successfully. Students should not hit avoidable errors from missing files, stale package names, or broken paths before they have written any training code.

5.4 Kaggle Workflow

A practical workflow is:

1. Fork or copy the starter repository, then develop and push your code to GitHub.
2. Clone the repository in Kaggle.
3. Run training in one notebook.
4. Save the Kaggle notebook version so that outputs are preserved.
5. Attach the training notebook outputs as an input dataset to a separate generation notebook.
6. Attach generation outputs as an input dataset to a separate evaluation notebook.

This separation avoids rerunning expensive training when debugging evaluation.

5.5 Saving Work While Kaggle Jobs Run

Kaggle notebook sessions are temporary. Anything important must be written under `/kaggle/working` and then preserved by saving a notebook version or creating an output dataset. The safest pattern is to make every long-running stage write durable artifacts as it goes, rather than only at the very end.

For training notebooks:

1. Write checkpoints, metrics, logs, and plots under a stable run directory such as `/kaggle/working/<repo_name>/outputs/<run_name>/`.
2. Save at least `best` and `final` checkpoints. Avoid saving too many full intermediate checkpoints because Kaggle disk is limited.
3. Write a small metrics file after every evaluation step, for example `metrics.jsonl`, so partial progress survives if the notebook disconnects.
4. When a training stage completes, use **Save Version** in Kaggle. Saved versions preserve the notebook outputs and can be attached to later notebooks as input datasets.
5. For extra safety, create a zip archive in `/kaggle/working`, for example `sft_run_outputs.zip` or `dpo_run_outputs.zip`. Zips are easier to attach, download, and audit.

For generation and evaluation notebooks:

1. Write generation JSON files immediately after each model finishes.
2. Write pairwise JSONL files before judging, not inside the judging loop.
3. During model-based judging, append or rewrite judgment files after every batch so the run can resume from existing outputs.
4. Keep failure logs, retry counts, and summary files in the same output directory.

A useful mental model is: every notebook should be restartable. If Kaggle disconnects halfway through, you should be able to attach the previous output dataset, skip completed work, and continue from the last saved artifact.

6 Model and Data

6.1 Model

Use `Qwen/Qwen2.5-0.5B` as the base model. This model is small enough to train on Kaggle T4 GPUs, while still being large enough to show meaningful differences between base, SFT, and DPO behavior.

6.2 SFT Data

Use an instruction-response dataset such as UltraChat-style data. Each example should be converted into a single-turn instruction and response.

Recommended run size:

Split	Examples	Purpose
Train	30,000	SFT optimization
Validation	1,000	validation loss

6.3 Preference Data

The assignment uses three preference datasets, each with examples containing a prompt, a chosen response, and a rejected response:

Track	Data	Recommended train size
HH-RLHF DPO	Helpful/harmless preference pairs. Use this for the safety-oriented from-scratch DPO run.	15,000
UltraFeedback DPO	General instruction-following preference pairs. Use this for the broad helpfulness from-scratch DPO run.	15,000
LoRA persona DPO	Provided small preference subset in `data/persona_lora/ramsay_style_preferences.json`. Use this with TRL `DPOTrainer` and LoRA.	provided subset

The reference setup uses single-turn examples. Use 1,000 validation pairs for each from-scratch DPO run when available, and filter out multi-turn conversations unless you explicitly implement a multi-turn prompt format.

6.4 Evaluation Prompts

Use fixed evaluation sets so that models are compared on the same prompts:

Training track	Evaluation set
HH-RLHF DPO	The synthetic helpful/harmless evaluation set provided with the assignment.
UltraFeedback DPO	The fixed AlpacaEval subset generated with seed 42. The default is 300 prompts.
LoRA persona DPO	A small fixed style-evaluation prompt set covering cooking, coding, general help, and safety-sensitive prompts.

Save the exact subsets used for all comparisons. To reproduce the default AlpacaEval evaluation subset, use 300 prompts with seed 42.

7 Task 1: Repository and Kaggle Setup

Create a GitHub repository and a Kaggle notebook workflow. Your setup should:

1. Clone the starter repository, or your fork of it, in Kaggle.
2. Install dependencies without reinstalling PyTorch.
3. Confirm that two T4 GPUs are available.
4. Print package versions for PyTorch, Transformers, and Datasets.

8 Task 2: Baseline Generation

Create a fixed AlpacaEval subset and generate baseline responses from `Qwen/Qwen2.5-0.5B`.

Your generation script should save a JSON file with entries containing at least:

```
{
  "id": "...",
  "instruction": "...",
  "output": "...",
  "generator": "qwen2.5-0.5b-base",
  "dataset": "alpaca_eval"
}
```

Use greedy decoding for the main comparison unless you justify another choice. Small models sometimes fall into repetitive loops; if this happens, you may use deterministic anti-repetition controls such as a modest repetition penalty or a no-repeat n-gram constraint. Document any decoding changes so SFT and DPO are compared fairly.

9 Task 3: Supervised Fine-Tuning

9.1 Implementation Requirements

Implement your own SFT training loop. Your implementation should include:

1. Prompt formatting for instruction-response examples.
2. Response-only loss masking.
3. Padding and attention masks.
4. Gradient accumulation.
5. Mixed precision using fp16 autocast on T4 GPUs.
6. FP32 trainable weights when using gradient scaling.
7. Gradient clipping.
8. Validation loss.
9. Saving `best` and `final` checkpoints.

Do not save frequent full intermediate checkpoints on Kaggle unless you have enough disk space. Saving only `best` and `final` is sufficient.

9.2 Suggested Hyperparameters

Hyperparameter	Value
Model	`Qwen/Qwen2.5-0.5B`
Context length	512
Train examples	30,000
Validation examples	1,000
Per-device batch size	2
Gradient accumulation	16
Effective batch size	32
Learning rate	2×10^{-5}
Warmup	3%
Schedule	cosine decay
Weight decay	0.1
Gradient clipping	1.0
Epochs	1

10 Task 4: Two From-Scratch DPO Runs

10.1 Implementation Requirements

Implement DPO yourself and use the same core implementation for both preference datasets. Your implementation should include:

1. Loading the SFT model as the trainable policy.
2. Loading a frozen copy of the SFT model as the reference model.
3. Computing response-only log probabilities for chosen and rejected responses.
4. Computing the DPO loss.
5. Tracking validation DPO loss, reward accuracy, and reward margin.
6. Saving the best checkpoint according to validation reward accuracy.

Kaggle Memory Tip

A memory-friendly setup on Kaggle T4x2 is to place the policy on `cuda:0` and the frozen reference on `cuda:1`. The trainable policy should remain in fp32 when using gradient scaling; the frozen reference may be loaded in fp16.

10.2 Required DPO Runs

After SFT, run two separate DPO trainings from the same SFT checkpoint:

1. **HH-RLHF DPO**: train on 15,000 helpful/harmless preference pairs and evaluate on the synthetic helpful/harmless set included with the assignment.
2. **UltraFeedback DPO**: train on 15,000 UltraFeedback-style preference pairs and evaluate on the fixed AlpacaEval subset using the same seed used for the base and SFT generations.

These are two different alignment experiments. HH-RLHF emphasizes safety and refusal behavior. UltraFeedback is broader and usually emphasizes general helpfulness, instruction-following, and response quality.

10.3 Reference Training Recipe

The guide configuration is:

1. Train one SFT checkpoint from `Qwen/Qwen2.5-0.5B` on 30,000 examples.
2. Start each DPO run from the SFT `best` checkpoint.
3. Load a second frozen copy of the SFT `best` checkpoint as the reference model.
4. Choose an appropriate prompt format and use it consistently for SFT, DPO, and evaluation. Document the exact template or chat-template settings in the report.
5. Use response-only log probabilities: prompt tokens and padding tokens must not contribute to the chosen or rejected sequence score.
6. Use $\beta = 0.1$ and a small fixed learning rate, 1×10^{-6}.
7. Use gradient accumulation so the effective batch size is 32 even though each device sees only one pair at a time.
8. Save the checkpoint with the best validation reward accuracy, plus a final checkpoint.

The reference DPO run uses a small fixed learning rate. You may try schedules such as warmup or cosine decay, but if you do, report the change clearly.

10.4 Suggested Hyperparameters

Hyperparameter	Value
Initial policy	SFT `best` checkpoint
Reference model	frozen SFT `best` checkpoint
Prompt format	student choice; keep it consistent across training and evaluation
Train pairs	15,000 per DPO run
Validation pairs	1,000 per DPO run
Context length	512
Maximum prompt length	256
Per-device batch size	1
Gradient accumulation	32
Effective batch size	32
Learning rate	1×10^{-6}, fixed
Learning-rate decay	none by default for DPO
β	0.1
Optimizer	RMSprop or AdamW
Gradient clipping	1.0
Epochs	1
Checkpoint selection	best validation reward accuracy

10.5 Metrics

For each validation pair, define the implicit rewards:

$$r_w = \beta \left(\log \pi_\theta(y_w \mid x) - \log \pi_{\text{ref}}(y_w \mid x) \right), \tag{7}$$

$$r_l = \beta \left(\log \pi_\theta(y_l \mid x) - \log \pi_{\text{ref}}(y_l \mid x) \right). \tag{8}$$

Reward accuracy is the fraction of validation examples where

$$r_w > r_l. \tag{9}$$

Reward margin is the mean value of

$$r_w - r_l. \tag{10}$$

If the policy and reference models are initially identical, every reward pair is tied: $r_w = r_l = 0$. Because reward accuracy uses the strict comparison $r_w > r_l$, its exact initial value is then 0, not 0.5. After the policy begins to move away from the reference, an accuracy near 0.5 with a margin near 0 suggests no consistent preference direction. Higher reward accuracy with a positive margin suggests the policy is learning to prefer chosen responses.

11 Task 5: Idiot Sandwich — TRL DPOTrainer With LoRA Persona Tuning

This task uses TRL's `DPOTrainer` and PEFT/LoRA to train a small instruction model on the provided persona preference subset. Unlike Task 4, this task **does** allow TRL and PEFT. Yes, it is a small style-steering experiment inspired by the following meme. Bon appétit.



Model	`HuggingFaceTB/SmolLM2-360M-Instruct`
Dataset	`data/persona_lora/ramsay_style_preferences.json`

Each row contains `prompt`, `chosen`, and `rejected`. Format it into the conversational structure expected by TRL.

Do not treat the LoRA setup as a copy-paste exercise. Read the PEFT documentation and the TRL DPOTrainer documentation, then choose an appropriate LoRA configuration for the model. Your report should briefly justify the LoRA rank, target modules, learning rate, batch size, and number of epochs you used.

Evaluate whether the LoRA-DPO model became more stylistically faithful without becoming less useful or less safe.

12 Task 6: Generate Evaluation Outputs

Before launching full-subset generation, run a small sanity check on 2–5 prompts. Inspect the responses manually to confirm that the checkpoint path, tokenizer, prompt format, EOS behavior, and generation length are reasonable.

Generate outputs from at least:

1. the SFT model;
2. the DPO model being compared against it.

For the UltraFeedback DPO run, use the fixed AlpacaEval subset generated with 300 prompts and seed 42. For the HH-RLHF DPO run, use the provided synthetic helpful/harmless evaluation set.

Build pairwise comparison files for **DPO versus SFT**. Randomize response order in each pairwise file and preserve the mapping from response position to model identity as hidden metadata. The judge prompt must label the outputs only as Response A and Response B; it must not reveal model names, checkpoint names, or whether an output came from SFT or DPO. You may generate base-model outputs for your own analysis, but the required comparison is DPO vs SFT.

Generation Coherence Tip

If outputs from the small models become repetitive, you may try deterministic anti-repetition settings such as a repetition penalty or no-repeat n-gram constraint. Use the same decoding settings for SFT and DPO in a comparison so the judge is evaluating the models, not different decoding policies.

13 Task 7: Evaluation With Kaggle Models

Use Kaggle model access through `kaggle_benchmarks` or an equivalent Kaggle-supported model interface. A suitable judge model is:

`google/gemini-3.6-flash`

If that model is unavailable in your Kaggle environment, you may use another supported Kaggle model-proxy judge. Record the judge model you used in your report.

13.1 Kaggle Benchmark Setup

For model-based evaluation, create or use a Kaggle notebook that is configured as a **utility benchmark**. This is important because Kaggle's benchmark/model-proxy environment allows free model calls to supported judge models such as `google/gemini-3.6-flash`. A practical setup is:

1. Save generation outputs from the training/generation notebook as a Kaggle output dataset.
2. Create a separate evaluation notebook and attach the generation-output dataset as an input.
3. Enable the Kaggle benchmark/model access needed by `kaggle_benchmarks`.
4. Run judging from saved pairwise JSONL files only. Do not regenerate model outputs inside the judge notebook.
5. Write judgment files back to `/kaggle/working` and save the evaluation notebook version when judging completes.

The evaluation notebook should be treated as a resumable batch job. It should not assume that all judge calls succeed on the first attempt.

13.2 Evaluator Reliability Requirements

Your evaluator should:

1. Read saved pairwise JSONL files.
2. Judge in small batches, such as 5 pairs per request.
3. Checkpoint judgments after each batch.
4. Sleep at least 5 seconds between judge request attempts to avoid overloading the model proxy or receiving request-denied errors.
5. Retry failed batches multiple times, for example up to 3 attempts per batch.
6. Log unresolved failures to a separate JSONL failure file.
7. Resume from existing judgments when possible so reruns do not spend calls on completed pairs.
8. Summarize win, loss, tie, unknown, and failure rates.

A representative command is:

```
python scripts/judge_pairwise_kaggle.py \
  --pairs_path outputs/eval/dpo_vs_sft_pairs.jsonl \
  --output_path outputs/eval/dpo_vs_sft_judged.jsonl \
  --judge_model google/gemini-3.6-flash \
  --batch_size 5 \
  --sleep_seconds 5 \
  --max_retries 3 \
  --reuse_judgments
```

The judge should be instructed to compare responses based on helpfulness, correctness, instruction-following, concision when appropriate, formatting, and absence of hallucinations or unsafe content. For safety-focused evaluation sets, the prompt should also tell the judge to reward appropriate refusal, privacy protection, calibrated uncertainty, and safe redirection. Keep model identities out of the judge-facing prompt and translate the returned A/B decision back to the stored model identity only after judging.

Do Not Hammer the Judge Model

Large, fast request loops are more likely to fail or be denied. Prefer small batches, a 5-second delay, retries with logging, and resumable output files. This is slower but much more reliable.

14 Task 8: Final Report

Your final report must be no more than 8 pages, including figures, tables, and references. It should be concise while making the experimental choices and comparisons auditable. Organize it using the following structure.

14.1 Repository, Models, and Data

Provide the GitHub repository link. Identify the base checkpoint, the datasets and subsets used for each training track, the number of training and validation examples, and the fixed evaluation sets. State the random seeds and any filtering or prompt formatting choices that could affect reproducibility.

14.2 Experimental Configuration

Include one compact table with a row for SFT, HH-RLHF DPO, UltraFeedback DPO, and LoRA-DPO. Report at least:

- trainable parameter strategy: full fine-tuning or LoRA, and LoRA rank, alpha, dropout, and target modules where applicable;
- optimizer, peak learning rate, scheduler, warmup, weight decay, and gradient clipping;
- per-device batch size, gradient accumulation, effective global batch size, precision, maximum sequence length, epochs or optimizer steps, and seed;
- DPO beta and the exact reference checkpoint for every DPO run;
- generation settings used during evaluation, including temperature, top-p, maximum new tokens, repetition/length penalties, and any chat template.

Do not report only the guide configuration. Report the values actually used for the submitted checkpoints.

14.3 Parameter Choices and Deviations

For every meaningful change from the guide configuration, state the guide value, your final value, and why you changed it. Tie the explanation to concrete evidence where possible: memory limits, instability, validation behavior, reward margin, a short pilot run, or output quality. Explain what changed after the adjustment and whether it improved, worsened, or did not materially affect performance. If you tried multiple settings, a small comparison table is preferred over a chronological diary.

Resource-driven changes and performance-driven changes should be identified separately. For example, reducing batch size to avoid out-of-memory errors is different from changing beta because preference accuracy saturated. Both are valid, but they support different conclusions.

14.4 Training Behavior

Show readable plots for the training runs. Include training and validation loss for SFT where available, and DPO loss plus at least one preference diagnostic such as reward accuracy or reward margin for DPO. Label axes, runs, checkpoints, and smoothing. Briefly explain convergence, instability, overfitting, or the checkpoint-selection decision; do not infer final response quality from training loss alone.

14.5 DPO-versus-SFT Results

For each DPO track, compare its submitted checkpoint against the appropriate SFT baseline on the same prompts using the same generation settings and judging procedure. Report sample count,

wins, ties, losses, win rate with and without ties if used, judge model, failures/retries, and the handling of invalid judgments. Keep prompt order and evaluation settings fixed so that the comparison isolates the model checkpoint as much as possible.

Include the LoRA persona result separately because style alignment is a different target from general helpfulness or safety. Do not combine unlike evaluation sets into a single headline score.

14.6 Interpretation and Error Analysis

Answer explicitly: which model performed better on each evaluation set, by how much, and why do you think that happened? Distinguish measured observations from your interpretation. Relate the result to the preference dataset, SFT baseline, beta, training duration, reward diagnostics, and visible response behavior. Discuss whether the effect is consistent across prompt types or concentrated in areas such as safety refusals, factuality, instruction following, verbosity, or persona style.

Include a few representative examples where DPO improves over SFT and where it regresses or changes little. Use these examples to support the aggregate metrics, not replace them. Plausible explanations include preference-data mismatch, noisy pairs, over-optimization, insufficient training, a strong SFT baseline, or decoding sensitivity, but claims must be connected to evidence from your runs.

14.7 Limitations and Reproducibility

Summarize important limitations such as judge-model bias, small evaluation size, single-seed variance, failed runs, or compute constraints. Provide the paths to key configurations, metrics, generation files, and judged outputs in the repository so staff can reproduce the reported tables.

Keep the report direct. You do not need a long literature review, generation-throughput table, or exhaustive qualitative appendix. The priority is a fair, reproducible comparison and a technically grounded explanation of why performance changed.

15 Expected Runtime

For the recommended 30k SFT, two 15k DPO runs, LoRA-DPO, and 300-prompt evaluation setup, a rough estimate on Kaggle T4x2 is:

Stage	Estimated time
SFT training	1.5 hours
Each from-scratch DPO training	2.0–2.5 hours
LoRA-DPO training	0.1–0.3 hours
Generation	0.5–1.0 hours
Kaggle model judging	0.3–0.5 hours
Total	6.5–8.5 hours

Runtime Planning

Budget an eight- to nine-hour Kaggle session to be safe if you run every track end to end. Save notebook versions and archive important outputs as soon as a stage completes.

16 Submission Checklist

Submit:

1. GitHub repository URL for your completed project.
2. A short final report as PDF.
3. Training plots for SFT and DPO runs.
4. Evaluation summary comparing DPO against SFT.
5. A root-level `submission_adapter.py` connected to your implementation, with the public adapter tests passing.

17 Academic Integrity and Collaboration

You may discuss concepts, debugging strategies, and experimental observations with classmates. Your implementation and report must be your own. If you use external references, cite them. Do not submit code copied from public DPO implementations as your own. You may consult documentation for PyTorch, Hugging Face Transformers, Kaggle, and the DPO paper.

18 Resources

Useful Links

Hugging Face

Transformers docs

Qwen2.5-0.5B model card

PEFT / LoRA

PEFT docs

Preference Optimization

DPO paper

TRL docs

DPOTrainer guide

Kaggle

Notebooks docs

Models docs

GPU usage

References

1. Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., and Finn, C. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. NeurIPS, 2023.
2. Ouyang, L. et al. Training language models to follow instructions with human feedback. NeurIPS, 2022.
3. Dubois, Y. et al. Length-Controlled AlpacaEval: A Simple Way to Debias Automatic Evaluators. 2024.
4. Bai, Y. et al. Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback. 2022.

5. Hugging Face. Transformers documentation, TRL documentation, and PEFT documentation.

6. Kaggle. Notebooks documentation, Models documentation, and efficient GPU usage guide.